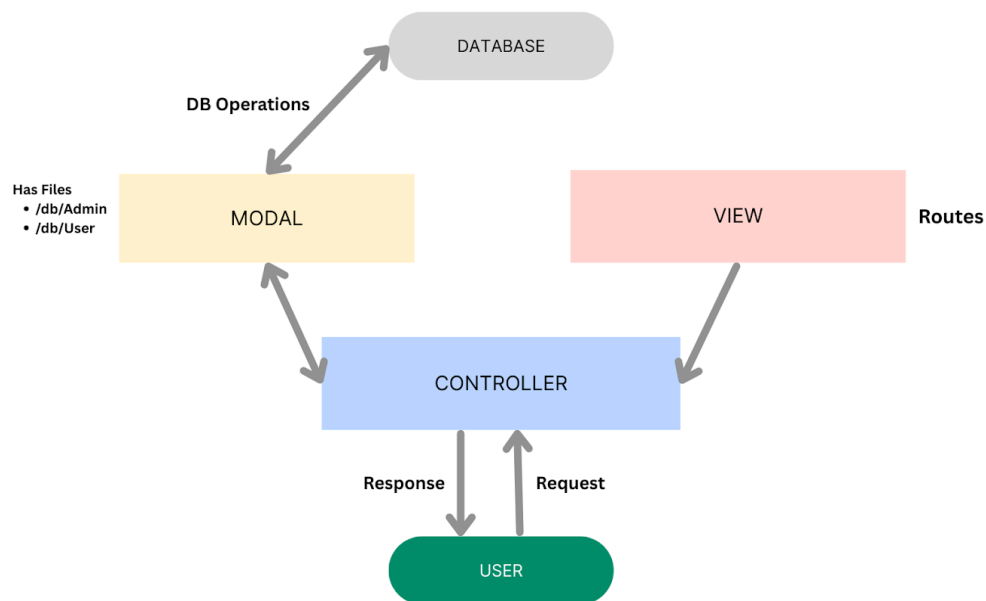


# Model-View-Controller (MVC) Architecture

**DarshanEase – Temple Darshan Ticket Booking System** follows the **Model-View-Controller (MVC)** architectural pattern. MVC is a widely used software design architecture that separates the application into three independent layers: **Model**, **View**, and **Controller**. This separation improves code organization, maintainability, scalability, and simplifies the development process.

## MVC Architecture Diagram



---

## Model Layer (Data Layer)

The **Model** layer is responsible for managing all application data and interacting with the MongoDB database. In the DarshanEase system, models are developed using **Mongoose**, which provides schema-based data modeling.

The Model layer stores and manages information such as:

- User Details
- Temple Information
- Darshan Slot Details
- Booking Information
- Payment Records

It performs all database operations such as creating, retrieving, updating, and deleting records while maintaining data integrity.

---

## Controller Layer (Business Logic Layer)

The **Controller** acts as the central processing unit of the application. It receives requests from the client through the API routes, validates the incoming data, executes the required business logic, communicates with the Model layer, and returns appropriate responses.

In the **DarshanEase** project, controllers are responsible for handling:

- User Registration and Login
- Temple Management
- Darshan Slot Management
- Ticket Booking
- Payment Processing
- Booking History
- User Profile Management

The Controller ensures smooth communication between the frontend interface and the database.

---

## View Layer (Routing/API Layer)

Since **DarshanEase** is developed as a **MERN Stack** application, the View layer in the backend is represented by the **API Routes**.

The routing layer defines REST API endpoints and forwards client requests to the appropriate controller methods.

Examples of API routes include:

- Authentication Routes
- User Routes
- Temple Routes
- Booking Routes
- Payment Routes
- Admin Routes

These routes handle HTTP methods such as **GET, POST, PUT, and DELETE** to perform various operations within the system.

---

# Database Layer

The MongoDB database serves as the central repository for storing all application data.

It maintains information related to:

- Registered Users
- Temple Details
- Darshan Schedules
- Available Slots
- Bookings
- Payments
- Ticket Information

The Model layer communicates directly with the database using Mongoose to perform CRUD (Create, Read, Update, Delete) operations.

---

## Working of MVC in DarshanEase

The workflow of the DarshanEase backend follows these steps:

1. The **User** sends a request from the frontend application.
  2. The request is received by the **API Routes (View Layer)**.
  3. The routes forward the request to the appropriate **Controller**.
  4. The **Controller** validates the request and performs the required business logic.
  5. The **Model** interacts with the **MongoDB Database** to retrieve or update data.
  6. The processed data is returned to the **Controller**.
  7. The **Controller** sends the response back to the frontend application.
- 

## Advantages of Using MVC in DarshanEase

- **Separation of Concerns:** Each layer has a specific responsibility, making the application easier to understand and maintain.
- **Modular Development:** Individual components such as Models, Controllers, and Routes can be developed and modified independently.
- **Scalability:** New modules like temple management, payment services, or additional booking features can be added without affecting existing functionality.
- **Code Reusability:** Business logic and database operations can be reused across different modules, reducing code duplication.

- **Easy Testing:** Models, controllers, and routes can be tested independently, improving software reliability.
- **Improved Collaboration:** Multiple developers can simultaneously work on different layers of the application without conflicts.
- **Maintainability:** The organized project structure makes debugging, updating, and extending the application much simpler.